

Report No. RP/01/2026 PL

Poznań University of Technology

Faculty of Computing and Telecommunications

Institute of Computer Science

Research Report

LEM — Location Encoding Model

Author: Piotr Augustyniak, M.Sc.

Report No.: RP/01/2026 PL

Version: 1.0

Status: Preprint — technical specification

Date: 13 September 2026

Research group led by dr hab. inż. Piotr Zwierzykowski, Professor of Poznań University of Technology

© 2026 Piotr Augustyniak. Report text and original tables and diagrams: CC BY 4.0.

Technical artefacts: Apache License 2.0. PUT logos, template class, and third-party materials are excluded.

Poznań, 2026

Abstract

This document specifies the Location Encoding Model (LEM), a compact, representation-independent information model for describing the physical location of an entity. LEM defines five components, their value domains, consistency constraints, and a deterministic canonical form. Text, JSON, binary TLV, and YANG representations map to the same semantic core; mappings to Civic Address and GeoJSON illustrate integration with established models. A lossy DNS-safe form is explicitly classified as a presentation alias rather than a semantically equivalent LEM representation. Application profiles may impose additional requirements without changing the core semantics. The specification also covers implementation, security, privacy, versioning, and machine-verifiable reference artefacts.

Table of Contents

Abstract	2
1. Introduction.....	7
1.1. Purpose of the LEM model.....	7
1.2. Scope of the document	7
1.3. Contribution of the document	8
1.4. Independence of semantics from representation.....	8
1.5. Structure of the document	8
1.6. Terminology and conventions.....	8
2. The LEM Semantic Model	10
2.1. Model components.....	10
2.1.1. Site	10
2.1.2. Building.....	10
2.1.3. Level	10
2.1.4. Reference Space.....	10
2.1.5. Relative Position.....	10
2.1.6. Formal data types of the components (normative).....	11
2.2. Relations between components.....	12
2.3. Classification of reference spaces	12
2.4. Canonical interpretation.....	12
2.4.1. Purpose of the canonical interpretation	12
2.4.2. Interpreting a description	13
2.4.3. Semantic equivalence.....	13
2.4.4. Partial descriptions	13
2.4.5. Boundary of interpretive responsibility	14
2.5. Model consistency rules.....	14
2.5.1. Unambiguity of interpretation.....	14
2.5.2. Interpretability of components	14
2.5.3. Consistency of spatial relations.....	14
2.5.4. Absence of semantic contradiction	14
2.5.5. Scope of implementation responsibility	14
2.5.6. Formal notation of consistency rules (informative)	15
2.6. Scope of responsibility of the LEM model.....	16
3. Representations and Mappings of the Model	17

3.1. Principles of model representation	17
3.2. Textual (human-readable) representation	17
3.3. Binary (compact/TLV) representation	18
3.3.1. Formal type registry (informative)	19
3.4. JSON representation	19
3.5. DNS-safe presentational alias	21
3.5.1. Formal label scheme (informative)	21
3.6. Mapping to existing information models	22
3.6.1. Mapping to YANG models	22
3.6.2. Mapping to Civic Address models	23
3.6.3. Mapping to geospatial models	24
3.7. Illustrative example (one location in all representations)	24
3.8. Examples of invalid location descriptions	25
4. Profiles and Applications of the LEM Model	26
4.1. Role of LEM profiles	26
4.2. Principles for defining profiles	26
4.3. Domain profiles and example applications	26
4.3.1. Network infrastructure	26
4.3.2. Infrastructure and CMDB management	27
4.3.3. Emergency location	27
4.3.4. Indoor positioning systems	29
4.3.5. Technical documentation	29
4.3.6. Automation and orchestration	29
4.3.7. Other domains	29
4.4. Constraints and implementation requirements of profiles	30
4.4.1. Minimum scope of information	30
4.4.2. Required components	30
4.4.3. Profile validation rules	30
4.4.4. Application-specific data schemas	30
4.5. Conformance of profiles with the LEM model	31
4.6. Extending and versioning profiles	31
4.6.1. Profile inheritance	31
4.6.2. Backward compatibility	31
4.6.3. Evolution of domain profiles	31

5. Implementation Considerations	33
5.1. Processing representations	33
5.2. Internal implementation representation	33
5.3. Data validation	33
5.4. Error handling	33
5.5. Versioning and implementation compatibility	34
6. Security and Privacy Considerations	35
6.1. Scope of the model's responsibility	35
6.2. Protecting information integrity	35
6.3. Separation of location from identification	35
6.4. Trust in the source of location information	36
6.5. Security profiles	36
6.6. Privacy of location information	36
6.7. Minimizing disclosed information	37
6.8. Semantic consistency of extensions and security	37
6.9. Consolidated threat model (informative)	37
7. Evolution, Extension, and Versioning of the LEM Model	39
7.1. Classification of model changes	39
7.1.1. Corrective change	39
7.1.2. Extending change	39
7.1.3. Semantic change	39
7.2. Versioning principles	39
7.3. Version compatibility	40
7.4. Principles for extending the model	40
7.5. Private and domain-specific extensions	40
7.6. Process for semantic changes	41
7.7. Handling unknown elements	41
7.7.1. Optional element	41
7.7.2. Critical element	41
7.8. Stability of the LEM model	41
8. Related Work and Positioning of the Model	42
8.1. Location information standards	42
8.2. Semantic building models	42
8.3. Scope comparison	42

8.4. Boundary of the claimed novelty	42
Appendix A. Glossary of LEM Terms.....	44
A.1. Normative terms	44
A.2. Glossary of terms.....	44
References	46

1. Introduction

1.1. Purpose of the LEM model

Location information is used by many technical systems operating across different application domains. Infrastructure management, automation, security, technical documentation, and location-based service systems all require an unambiguous description of the position of physical objects. In practice, this information is often represented using local conventions that depend on the system, vendor, or technology in use. Approaches commonly encountered include, among others:

- unstructured textual descriptions in CMDB systems, databases, or technical documentation,
- local device and space naming schemes,
- designations derived from architectural or organizational documentation,
- information maintained as operational knowledge held by specific individuals or teams.

Such descriptive approaches often fail to provide unambiguous, deterministic, machine-interpretable information. The lack of a shared semantic model results in, among other consequences, limited interoperability, hindered automation, and difficulties in maintaining consistency.

The Location Encoding Model (LEM) defines a shared semantic model whose purpose is to guarantee an unambiguous meaning for its components regardless of how they are represented or implemented. LEM defines the meaning of the information, while the manner in which that information is expressed in a particular format remains an independent layer.

1.2. Scope of the document

This document defines the Location Encoding Model (LEM) as an information model intended for describing location. The specification defines:

- the semantic model of location information,
- the components of the LEM model and their meaning,
- the rules for interpreting information, semantic equivalence, and model consistency,
- the rules for representing LEM information and for creating application profiles.

The document does not define data-interchange formats, communication protocols, location-measurement technologies, positioning algorithms, or requirements specific to any single environment.

LEM does not define mechanisms for object identification, for acquiring location information, or for methods of determining position. The model describes only the semantics of location information that is already available.

1.3. Contribution of the document

The contribution of LEM comprises four elements:

1. a small semantic core that separates the meaning of location from its representation;
2. a deterministic canonical form and explicit equivalence rules for partial descriptions;
3. consistent mappings to JSON, TLV, and YANG, together with integration mappings to Civic Address and GeoJSON;
4. a profile mechanism enabling domain-specific specialization without redefining the core model.

1.4. Independence of semantics from representation

The LEM model separates the meaning of information from the manner of its expression. Meaning is determined by the semantic model and the canonical interpretation. The same information may be expressed in different representations (textual, structural, binary) that are semantically equivalent if they lead to the same interpretation according to the rules of LEM.

1.5. Structure of the document

Chapter 2: Defines the LEM semantic model, its components, and the rules of interpretation.

Chapter 3: Defines the rules for representing the model in various data formats.

Chapter 4: Defines application profiles for domain-specific specialization.

Chapters 5–7: Discuss implementation, security, privacy, and the evolution of the model.

Chapter 8: Positions LEM in relation to existing standards and models.

1.6. Terminology and conventions

For the purposes of this document, the following definitions are adopted:

Information Model: An abstract model defining the structure, meaning, and relationships of elements.

Location Information: Information described using LEM components.

Model Component: An element of LEM that carries semantic meaning.

Reference Space: A space of reference for spatial interpretation.

Relative Position: A spatial relation with respect to the reference space.

Representation: A way of expressing information using a data format.

Canonical Interpretation: The unambiguous meaning derived from the LEM model.

LEM Profile: A specialization of the model that specifies requirements for a given application.

Keywords written in capital letters are used in accordance with BCP 14 [3, 10]: MUST corresponds to MUST, MUST NOT — to MUST NOT, SHOULD — to SHOULD, and MAY — to MAY.

Examples are illustrative in nature unless stated otherwise.

2. The LEM Semantic Model

2.1. Model components

Location is a logical composition of semantic components describing the physical location of an object. The individual components provide information about different aspects of location and may occur independently of one another, subject to the constraints specified in this document.

The LEM model defines a set of semantic components used to describe the physical location of an object. Each component represents a specific aspect of location information and has independent semantics. Components may be used singly or combined to achieve a greater level of detail in the location description.

Figure 2.1. The semantic core of the LEM model

[Diagram: Location entity composed of Site, Building, Level, Reference Space, and Relative Position (the latter defined with respect to Reference Space)]

Unless this specification states otherwise, the absence of a component indicates the absence of specified information, not a default value (see Section 2.4.4).

The following sections define the meaning of each component and the basic rules governing their use. Rules concerning dependencies between components, minimum requirements, semantic validation, cases of incomplete location descriptions, and information conflicts are defined jointly in Sections 2.4 (Canonical Interpretation) and 2.5 (Model Consistency Rules).

2.1.1. Site

Site — the highest level of the location hierarchy, defined by the implementation.

2.1.2. Building

Building — an element of the location hierarchy representing a single building or other delineated physical structure, defined by the implementation.

2.1.3. Level

Level — an element of the location hierarchy representing a level (storey) of space, defined by the implementation.

2.1.4. Reference Space

Reference Space — a uniquely identifiable space that serves as the point of reference for determining the relative position of an object.

2.1.5. Relative Position

Relative Position — a description of an object's position with respect to a defined reference space.

The meaning of each component and the relations between them is independent of the representation in which the component is recorded (see Chapter 3).

2.1.6. Formal data types of the components (normative)

The location of an entity is represented by five independent components. Each component has a defined type and semantics. Textual values contain between 1 and 256 Unicode code points. UTF-8 [18] is the encoding used by selected representations, and is not a measure of semantic length.

Site: - Type: string - Semantics: the textual name or identifier of the site/premises/campus within which the Building is located. - Requirement: an optional component in the base model; a profile may require its presence. - Design note (not part of the model): future integrations with external systems (CMDB, GIS) are recommended to distinguish the presentational value (name) from a stable identifier. The current model does not introduce this distinction.

Building: - Type: string - Semantics: the textual name or identifier of the building (or an equivalent structural object) located within the Site. - Requirement: an optional component in the base model. - Design note: analogous to Site.

Level: - Type: a Level structure with fields `kind` and `value`. - Semantics: - `kind = "numeric"` — `value` is required and must be a signed integer in the range from -2^{31} to $2^{31} - 1$. 0 denotes the ground floor level (a model convention, independent of local source numbering). Positive values denote above-ground storeys; negative values denote below-ground storeys. - `kind = "non-building"` — `value` must be null. This indicates that the entity is not located on any storey of a building (roof, ground-level car park, outdoor space, an area between buildings, etc.). Such a space is in that case still described via Reference Space (e.g., "Roof"), not via `value`. - Requirement: an optional component in the base model. - Prohibition: fractional values and types other than those specified above are not allowed. The mezzanine case remains unhandled by the base model.

Reference Space: - Type: string - Semantics: a named or otherwise designated space that serves as the location context for the entity — a room, hall, corridor, garage, parking area, roof, outdoor zone, or other spatially significant unit. Reference Space is not limited to the notion of "room." - Requirement: optional. An empty string is not allowed; the absence of information is expressed by the absence of the component.

Relative Position: - Type: string - Semantics: a description of the entity's position relative to the Reference Space (e.g., "by the window," "east wall," "above cabinet R1," "centrally," "near the entrance"). The value is a relational description — the model neither requires nor imposes a coordinate representation. - Requirement: an optional field. In accordance with Section 2.5.3, if present, it requires the presence of Reference Space in the same description.

Common rules:

- All textual values have a length of between 1 and 256 Unicode code points. The octet-length limit is a property of the representation.
- A valid description contains at least one core component.

- Components are independent — the absence of a value in one field does not invalidate the others (see Section 2.4.4).
- The model does not define a reference hierarchy (Site/Building as entities distinct from their textual values). Components serve solely a locational role.

2.2. Relations between components

The components of the LEM model are linked by relations (see Appendix A.6). Relations in the LEM model are directional (see Appendix A.13) — the order of the elements participating in a relation is part of its semantic meaning and must not be disregarded during interpretation.

For example, the relation `located_in(Object, ReferenceSpace)` has a different meaning from the inverse relation `contains(ReferenceSpace, Object)`. Preserving the direction of a relation is a condition for the correct interpretation of a location description, in accordance with the rules specified in Section 2.5.3.

2.3. Classification of reference spaces

The reference space determines the kind of space used as the reference for a spatial relation description within the LEM model.

The classification of reference spaces serves to organize possible categories of reference and does not define a closed set of space types. LEM does not specify an exhaustive list of reference-space classes. Implementations may define additional classes as required by a given application, provided the semantics of the LEM model are preserved.

The classification of reference spaces does not affect the meaning of location information. The meaning of a location description follows from the interpretation of the LEM model's components and the relations between them, in accordance with the rules specified in Section 2.4.

Membership of a reference space in a given class does not change the way the Relative Position relation, discussed in Section 2.5.3, is interpreted — the classification is organizational in character, not semantic.

2.4. Canonical interpretation

The canonical interpretation defines the deterministic form of a description after decoding a representation. It does not resolve whether different natural-language names denote the same object.

2.4.1. Purpose of the canonical interpretation

The canonical interpretation enables the comparison of valid representations independently of field order, formatting, or transport technology.

2.4.2. Interpreting a description

After a description D has been correctly decoded, its canonical form is the ordered partial tuple

$$C(D) = (S?, B?, L?, R?, P?),$$

where the symbols correspond, in order, to the components Site, Building, Level, Reference Space, and Relative Position, and the question mark denotes the possible absence of a value. Absence is neither an empty value nor a default value.

Textual values in $C(D)$ are the exact Unicode strings supplied by the representation. The base model does not perform automatic whitespace trimming, case conversion, translation, abbreviation expansion, or synonym recognition. A profile may define explicit normalization rules or stable identifiers, but these must precede the creation of $C(D)$ and must be applied identically by the implementations being compared.

2.4.3. Semantic equivalence

Two valid descriptions $D1$ and $D2$ are equivalent in the base model if and only if $C(D1) = C(D2)$, i.e., they contain exactly the same subset of components and identical values after decoding.

Semantic equivalence is not affected by:

- the order in which components appear in the location description,
- the manner of representing the information,
- the format in which the information is recorded,
- the presence of additional information not defined by the LEM model, provided it is separated from the core components and does not affect their decoding.

2.4.4. Partial descriptions

A location description may contain any non-empty subset of the components defined by the LEM model, with the proviso that Relative Position requires the presence of Reference Space.

The absence of a component indicates the absence of specified information within that scope and does not constitute a violation of the LEM model.

A partial description represents a smaller scope of location information than a description containing additional components, but remains a valid semantic description.

Requirements concerning the minimum scope of information for particular applications may be defined outside the LEM model — in particular, through application profiles (Chapter 4).

Privacy note: Even a partial description, limited to a small number of components, may in a given context constitute information sufficient for the indirect identification of a person or asset. The LEM model remains neutral with respect to this assessment (see Section 6.6), but

application profiles should take this aspect into account when determining the minimum and maximum scope of disclosed components.

2.4.5. Boundary of interpretive responsibility

LEM guarantees the unambiguity of the structure and role of its components, but not the ontological identity of arbitrary textual values. For example, “Room 201,” “201,” and “Sala 201” are different values until a shared profile or external registry defines them as aliases of the same identifier.

2.5. Model consistency rules

LEM defines rules of semantic consistency for location descriptions.

These rules specify the conditions that must be satisfied for a location description to remain compliant with the LEM model.

LEM does not define rules for the correctness of representation formats or methods for verifying source data. Consistency assessment applies exclusively to the meaning of information expressed through the model’s components.

2.5.1. Unambiguity of interpretation

Each core component may occur at most once. A representation that permits duplicates must reject them before a canonical form is created; it must not select one of the competing values.

2.5.2. Interpretability of components

Each component must satisfy the type and value domain specified in Section 2.1.6. A profile may narrow this domain but may not change the type or meaning of the component.

2.5.3. Consistency of spatial relations

Relative Position does not define a standalone location. Its presence without Reference Space causes the description to be rejected.

2.5.4. Absence of semantic contradiction

The base model detects structural contradictions: a duplicate component, a value outside its domain, an invalid Level *kind/value* pair, and Relative Position without Reference Space.

Domain-specific contradictions, e.g., a nonexistent room number in a given building, require a profile or an external registry.

2.5.5. Scope of implementation responsibility

LEM defines rules of semantic consistency for the location information model.

Requirements concerning the completeness of information, domain-specific constraints, and additional correctness conditions may be defined outside the LEM model.

Such requirements do not change the base semantics of the components or the interpretation rules defined by LEM.

2.5.6. Formal notation of consistency rules (informative)

Choice of notation and rationale. The following section adopts a two-layer approach: (1) a structural skeleton in the form of a JSON Schema, Draft 2020-12 [9], defining the shape and types of a valid LEM location description in its JSON representation, and (2) pseudocode for those consistency rules that JSON Schema is, in principle, unable to express. This arrangement was chosen for three reasons:

1. **Tooling availability:** For JSON Schema, ready-made, widely used validators exist in nearly every programming language.
2. **Consistency with the rest of the document:** Section 4.4.4 already introduces JSON Schema as a mechanism for expressing profile requirements.
3. **Deliberately limited scope:** JSON Schema (Draft 2020-12) supports cross-field conditions; rules that go beyond document structure remain in pseudocode.

(1) Structural skeleton (JSON Schema, informative)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://w3id.org/lem/1.0/schema/core",
  "title": "LEM -- structural skeleton of a location description",
  "type": "object",
  "properties": {
    "site": { "type": "string", "minLength": 1, "maxLength": 256 },
    "building": { "type": "string", "minLength": 1, "maxLength": 256 },
    "level": {
      "type": "object",
      "properties": {
        "kind": { "enum": ["numeric", "non-building"] },
        "value": { "type": ["integer", "null"],
          "minimum": -2147483648, "maximum": 2147483647 }
      },
      "required": ["kind", "value"],
      "if": { "properties": { "kind": { "const": "numeric" } } },
      "then": { "properties": { "value": { "type": "integer" } } },
      "else": { "properties": { "value": { "type": "null" } } }
    },
    "referenceSpace": { "type": "string", "minLength": 1, "maxLength": 256 },
    "relativePosition": { "type": "string", "minLength": 1, "maxLength": 256 }
  },
  "additionalProperties": true,
  "anyOf": [
    { "required": ["site"] }, { "required": ["building"] },
    { "required": ["level"] }, { "required": ["referenceSpace"] },
    { "required": ["relativePosition"] }
  ],
  "dependentRequired": {
    "relativePosition": ["referenceSpace"]
  }
}
```

The if/then/else construct for level encodes the rule: for kind = "numeric", the value field must be an integer; for kind = "non-building", it must be null. additionalProperties: true reflects the “Format autonomy” principle (Chapter 3.1). minProperties: 1 reflects the fact that an empty location description does not constitute a valid partial description (see Section 2.4.4).

Verification note: The full schema in the file `schemas/lem-core.schema.json`, together with two profile schemas, was validated against the JSON Schema Draft 2020-12 meta-schema using the `jsonschema 4.26.0` library. The package includes 20 positive and negative test vectors covering, among other things, int32 boundaries, the Level structure, the Relative Position/Reference Space dependency, empty descriptions, and profile requirements. The script `tests/validate.py` reproduces the validation result.

(2) Pseudocode for constructing and comparing the canonical form

The pseudocode below does not attempt to guess the meaning of arbitrary textual values.

```
decode(R):
    reject duplicate core components in representation R
    D := map representation fields to the five LEM components
    require D contains at least one core component
    require all values satisfy their LEM value domains
    require RelativePosition not in D or ReferenceSpace in D
    return D

canonical(R):
    D := decode(R)
    return tuple(D.Site?, D.Building?, D.Level?,
                D.ReferenceSpace?, D.RelativePosition?)

equivalent(R1, R2) <==> canonical(R1) == canonical(R2)
```

2.6. Scope of responsibility of the LEM model

The LEM model defines exclusively the semantics of location information. LEM does NOT define: identification mechanisms, data-acquisition methods, measurement technologies, management systems, or decision-making processes.

3. Representations and Mappings of the Model

3.1. Principles of model representation

The LEM model defines the meaning of location information independently of the manner in which it is recorded or transmitted. A representation of the model specifies how the components of the LEM model are expressed in a particular data format or information-exchange mechanism.

All representations of the LEM model are subject to the following principles:

- **Invariance of semantics:** A representation must faithfully reflect the meaning of the components defined in Chapter 2. A representation must not introduce new semantic elements or change the definitions of existing components.
- **Unambiguity of mapping:** Every representation must allow for an unambiguous mapping of the recorded data onto the components of the LEM model.
- **Consistency with the canonical interpretation:** Every representation of LEM information must lead to an interpretation consistent with the canonical interpretation defined in Section 2.4.5. A representation that conflicts with the canonical interpretation is not a valid representation of the LEM model.
- **Equivalence of representations:** Different representations are semantically equivalent if they have the same semantic interpretation in accordance with the rules specified in Section 2.4.3.
- **Format autonomy:** A representation may contain additional information, including technical metadata, provided that this information does not affect the interpretation of the LEM model's components.
- **No extension of the model:** A representation or mapping must not extend the meaning of the LEM model or introduce new location-information components. This also applies to technical constraints imposed by an external format or system — a technical constraint never constitutes an extension of the model's semantics (see Section 4.5).

3.2. Textual (human-readable) representation

The textual representation is used to present location information defined by the LEM model in a human-readable form.

The textual representation presents the components of the LEM model in a form intended for interpretation by a human reader. The layout of information, the order in which components appear, the separators used, or graphical formatting do not affect the meaning of the location description.

The meaning of the textual representation derives solely from the components of the LEM model that are present and their semantic interpretation.

Informative example:

Site: Main Campus
Building: Engineering Center
Level: 2
Reference Space: Room 201
Relative Position: East side

Figure 3.1. The semantic core of LEM and its representations and mappings

[Diagram: LEM core mapped to JSON, TLV, YANG, Civic/PIDF-LO, GeoJSON, and the DNS-safe presentational alias]

The above listing presents location information in a form readable by a human. The specific form of presentation is not part of the LEM semantic model.

3.3. Binary (compact/TLV) representation

The binary representation allows LEM information to be expressed as a compact data structure intended for exchange or storage.

The binary encoding mechanism does not define the meaning of location information. The meaning of each element of the representation derives from its mapping onto the components of the LEM model.

The binary representation may make use of mechanisms such as Type-Length-Value (TLV) — an encoding pattern formally recognizable, among other places, from ITU-T Recommendation X.690 [8] (see Appendix B) — however, the manner of assigning types, encoding values, and organizing the data structure belongs to a particular representation specification or implementation profile.

In the binary representation:

- the type of a representation element corresponds to a component of the LEM model,
- the value of a representation element corresponds to the value of the given component,
- the manner of value encoding does not change the meaning of the component.

A valid binary representation must lead to the same semantic interpretation that would result from any other valid representation of LEM information.

Informative example (TLV encoding profile defined below: T = 1 octet, L = 2 octets big-endian, V = value; Level uses one octet for `kind` plus four octets of int32 for the numeric value):

Type (T)	Component name	Length (L)	Value (V)
0x01	Site	0x000B	"Main Campus"
0x02	Building	0x0012	"Engineering Center"
0x03	Level	0x0005	0x01 00000002
0x04	Reference Space	0x0007	"Room201"
0x05	Relative Position	0x0009	"East side"

Octet stream (hexadecimal notation):

```
01 00 0B 4D 61 69 6E 20 43 61 6D 70 75 73
02 00 12 45 6E 67 69 6E 65 65 72 69 6E 67 20 43 65 6E 74 65 72
03 00 05 01 00 00 00 02
04 00 07 52 6F 6F 6D 32 30 31
05 00 09 45 61 73 74 20 73 69 64 65
```

The above assignment of type values (T) to a specific component (e.g., 0x03 → Level) is an element of a particular representation specification or implementation profile, and not part of the LEM semantic model — a different binary profile may assign the same components to different T values while remaining semantically equivalent (see Section 2.4.3), provided the mapping onto the model’s components is unambiguous.

3.3.1. Formal type registry (informative)

The registry below formalizes the example above:

1. The L field has 2 octets, allowing the full range of UTF-8 representation lengths to be described.
2. The encoding of Level reflects the `kind+value` structure and the shared int32 domain.
3. Relative Position is a plain UTF-8 string, with no subtype.

T (1 octet)	Component	Value encoding (V)
0x01	Site	UTF-8 [18], 1–1024 octets; after decoding, 1–256 Unicode code points
0x02	Building	UTF-8 [18], 1–1024 octets; after decoding, 1–256 Unicode code points
0x03	Level	1 octet <code>kind</code> (0x01=numeric, 0x02=non-building) + for numeric, exactly 4 octets of a signed int32, two’s complement, big-endian; for non-building, no further octets
0x04	Reference Space	UTF-8 [18], 1–1024 octets; after decoding, 1–256 Unicode code points
0x05	Relative Position	UTF-8 [18], 1–1024 octets; after decoding, 1–256 Unicode code points

Header format: T (1 octet) + L (2 octets, big-endian, length of V in octets) + V.

Consistency rules 2.5.1–2.5.4 apply to the set of decoded components in exactly the same way as for the JSON representation (see Section 2.5.6).

3.4. JSON representation

The JSON representation allows LEM information to be expressed using a JSON data structure. JSON is exclusively a means of representing the components of the LEM model and does not define its own semantics for location information.

The representation uses JSON conformant with [4]. The parser **MUST** reject objects containing duplicate core property names before the canonical form is created.

The JSON structure must allow the elements of the representation to be unambiguously mapped onto the components of the LEM model.

Property names, object structure, and the manner of value encoding are elements of the representation and must not change the meaning of the components defined in the LEM model.

Two JSON documents are semantically equivalent if their interpretation leads to the same canonical interpretation of the LEM model.

Differences in:

- the order of JSON object properties,
- the document's formatting,
- the technical representation of values, provided the equivalence of meaning is preserved,

do not affect the semantic equivalence of the information.

Informative example:

```
{
  "site": "Main Campus",
  "building": "Engineering Center",
  "level": { "kind": "numeric", "value": 2 },
  "referenceSpace": "Room201",
  "relativePosition": "East side"
}
```

The following document is semantically equivalent to the one above (different property order, different formatting):

```
{
  "level": { "value": 2, "kind": "numeric" },
  "site": "Main Campus",
  "relativePosition": "East side",
  "referenceSpace": "Room201",
  "building": "Engineering Center"
}
```

The property names used above (site, building, level, etc.) are an element of a specific representation — a profile or implementation schema MAY use different names (see Sections 3.1, 4.4.4), provided the mapping onto the components of the LEM model remains unambiguous.

Informative example — the non-building case (see Section 2.1.6): an object located on the roof of a building:

```
{
  "site": "Main Campus",
  "building": "Engineering Center",
  "level": { "kind": "non-building", "value": null },
  "referenceSpace": "Roof"
}
```

3.5. DNS-safe presentational alias

The DNS-safe alias allows a selected subset of LEM information to be presented in environments subject to constraints characteristic of DNS names [11].

Technical constraints arising from DNS name-formation rules do not define the meaning of location information.

The meaning of the DNS-safe representation derives solely from the mapping of the elements used onto the components of the LEM model.

The DNS-safe alias may present only a subset of the LEM model's components. The inability to express a given component in a particular environment does not change the meaning of the model but signifies a loss of information within the alias.

The text canonicalization defined below is lossy; therefore DNS-safe is NOT an equivalent or reversible representation of LEM. It is exclusively a presentational alias. A system that needs to reconstruct the canonical form MUST transport the full LEM representation over a separate channel, or use a separate, lossless encoding with an unambiguous decoding scheme.

Informative example:

level2.room201.eng-center.main-campus.loc.example.

3.5.1. Formal label scheme (informative)

The formal scheme introduces explicit, short prefixes in each label, independent of its position within the name:

Prefix	Component	Encoding of the label core
s-	Site	Canonicalized string
b-	Building	Canonicalized string
l-	Level	1-<N> for kind="numeric", non-negative value; 1-n<N> for a negative value; 1-nb for kind="non-building"
r-	Reference Space	Canonicalized string

Relative Position is never represented in DNS-safe form.

Structure of the full name (ABNF notation, per [7]):

```
lem-dns-name = *(component-label ".") suffix "."
component-label = prefix "-" label-core
prefix          = "s" / "b" / "l" / "r"
label-core      = 1*61(ALPHA / DIGIT / "-")
suffix          = 1*(dns-label ".") dns-label
```

Canonicalization of a textual value (Site, Building, Reference Space) into a label core: conversion to lowercase, replacement of spaces and other characters outside [a-z0-9] with -

, collapsing of consecutive - characters into one, removal of - from the edges. This canonicalization is lossy.

Note: Punycode [6] encodes Unicode in DNS labels, but its use alone does not restore information lost through the removal of separators, the component scope, or the omission of Relative Position. A lossless variant requires a separate specification.

Formal example:

```
s-main-campus.b-engineering-center.l-2.r-room201.loc.example.
```

Example with a negative Level:

```
s-main-campus.l-n1.r-parking-p1.loc.example.
```

Example with non-building (roof):

```
s-main-campus.b-engineering-center.l-nb.r-roof.loc.example.
```

3.6. Mapping to existing information models

Existing information models may be used to express selected aspects of LEM information through defined semantic mappings.

Mapping to an external information model does not mean replacing the LEM model or extending its meaning.

The external model serves as a source of representation for particular components, whose meaning must remain consistent with the LEM interpretation.

Components of the external model that have no counterpart in the LEM model remain additional information and are not part of the LEM location description.

3.6.1. Mapping to YANG models

YANG models may be used as a structural representation of LEM components in network- and system-management environments.

Informative example — an illustrative YANG module [2]:

```
module example-lem-location {
  namespace "urn:example:lem-location";
  prefix "lem";

  description
    "Illustrative mapping of the LEM semantic model's components (Chapter 2)
    onto YANG data nodes. Informative only (see Section 3.6.1).
    The Level structure reflects Section 2.1.6 (kind/value).";

  grouping lem-location {
    description "Group of nodes corresponding to the LEM model's components (see 2.1).";

    leaf site {
      type string { length "1..256"; }
      description "Mapping of the LEM component: Site (see 2.1.6).";
    }
  }
}
```

```

leaf building {
  type string { length "1..256"; }
  description "Mapping of the LEM component: Building (see 2.1.6).";
}

container level {
  description
    "Mapping of the LEM component: Level (see 2.1.6). A kind/value
    structure instead of a plain integer.";

  leaf kind {
    type enumeration {
      enum numeric { description "A numbered storey."; }
      enum non-building { description "Roof, ground-level car park, outdoor space, etc."; }
    }
    mandatory true;
  }

  leaf value {
    type int32;
    when "../kind = 'numeric'";
    description "Required and integer for kind=numeric; absent for kind=non-building.";
  }

  must "(kind = 'numeric' and value) or (kind = 'non-building' and not(value))" {
    error-message "Per 2.1.6: value is required for kind=numeric, and not allowed for kind=non-
building.";
  }
}

leaf reference-space {
  type string { length "1..256"; }
  description "Mapping of the LEM component: Reference Space (see 2.1.6).";
}

leaf relative-position {
  type string { length "1..256"; }
  description "Mapping of the LEM component: Relative Position (see 2.1.6) – a relational
description.";

  must "../reference-space" {
    error-message "Relative Position present without Reference Space violates LEM 2.5.3.";
  }
}

container device-location {
  description "Example configuration node using the lem-location grouping.";
  uses lem-location;
}

```

3.6.2. Mapping to Civic Address models

Civic Address models may be used to represent selected components describing the address-related aspects of location information.

Informative example — a PIDF-LO `civicAddress` document [16]:

```

<civicAddress xml:lang="en-US"
  xmlns="urn:ietf:params:xml:ns:pidf:geopriv10:civicAddr">

```

```

<country>US</country>
<NAM>Main Campus</NAM>
<BLD>Engineering Center</BLD>
<FLR>2</FLR>
<ROOM>Room201</ROOM>
<LOC>East side</LOC>
</civicAddress>

```

CAtype	LEM component	Nature of the mapping
BLD (27)	Building	Direct mapping.
FLR (28)	Level, kind="numeric"	The mapping applies only to the kind="numeric" case.
(none directly)	Level, kind="non-building"	Civic Address has no dedicated element.
ROOM (23)	Reference Space	Direct mapping.
NAM (22)	Site (interpretive)	Interpretive.
LOC (1)	Relative Position	Semantically close, but not identical.

Table 3.1. Mapping of selected Civic Address types [15] onto LEM components

3.6.3. Mapping to geospatial models

Geospatial models may be used to express the spatial aspects of location information, in particular coordinates or spatial references.

Informative example — a GeoJSON object [5]:

```

{
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [-73.961452, 40.808058]
  },
  "properties": {
    "lem:site": "Main Campus",
    "lem:building": "Engineering Center",
    "lem:level": { "kind": "numeric", "value": 2 },
    "lem:referenceSpace": "Room201",
    "lem:relativePosition": "East side"
  }
}

```

The `lem:` prefix in property names is an informative convention. The coordinate order [longitude, latitude] is required by [5] §4.

3.7. Illustrative example (one location in all representations)

The following example presents the same location information expressed in turn in every representation. All of the listings below **MUST** lead to the same canonical interpretation (see Section 2.4.5).

Location described: an object located in the building “Engineering Center” on the “Main Campus” campus, on the second storey, in room “Room201,” on the east side of that room.

1. Textual representation (Section 3.2):

Site: Main Campus
Building: Engineering Center
Level: 2 (numeric)
Reference Space: Room201
Relative Position: East side

2. Binary/TLV representation (Section 3.3, per the formal registry in 3.3.1 — with the L field on 2 octets):

```
01 00 0B 4D 61 69 6E 20 43 61 6D 70 75 73
02 00 12 45 6E 67 69 6E 65 65 72 69 6E 67 20 43 65 6E 74 65 72
03 00 05 01 00 00 00 02
04 00 07 52 6F 6F 6D 32 30 31
05 00 09 45 61 73 74 20 73 69 64 65
```

3. JSON representation (Section 3.4):

```
{
  "site": "Main Campus",
  "building": "Engineering Center",
  "level": { "kind": "numeric", "value": 2 },
  "referenceSpace": "Room201",
  "relativePosition": "East side"
}
```

4. DNS-safe presentational alias (Section 3.5.1) — a lossy encoding:

s-main-campus.b-engineering-center.l-2.r-room201.loc.example.

3.8. Examples of invalid location descriptions

Example 1 — violation of spatial-relation consistency (2.5.3):

```
{
  "site": "Main Campus",
  "building": "Engineering Center",
  "level": { "kind": "numeric", "value": 2 },
  "relativePosition": "East side"
}
```

The Relative Position component is present, but Reference Space is missing.

Example 2 — an empty location description (2.4.4):

```
{}
```

Example 3 — violation of the Level structure (2.1.6):

```
{
  "site": "Main Campus",
  "level": { "kind": "non-building", "value": 3 }
}
```

4. Profiles and Applications of the LEM Model

4.1. Role of LEM profiles

A LEM profile defines how the LEM model is applied within a particular environment or domain. A profile MAY restrict the scope of components used and specify additional implementation requirements, but it does NOT EXTEND the LEM semantic model and does NOT DEFINE new location-information components. A profile specifies how the existing components of the LEM model are used in a particular application context. A profile does not change the meaning of the model's components or the rules for their interpretation.

4.2. Principles for defining profiles

A LEM profile MUST satisfy the following principles:

- **Semantic conformance:** A profile MUST NOT change the meaning of the LEM model's components or define an alternative interpretation of the information.
- **Scope restriction:** A profile MAY restrict the scope of components used, but MUST NOT remove their meaning from the base model.
- **Unambiguity of requirements:** Requirements specified by a profile MUST indicate which components are required, optional, or unused in a given application. The presence status of a component is not the same as the rules for handling unknown extensions from Section 7.7.
- **Representation independence:** A profile MAY specify requirements regarding data representation, but MUST NOT make the meaning of the information dependent on a particular recording format.

4.3. Domain profiles and example applications

The following subsections describe example domains in which the LEM model may be applied. Each description covers both the role of the profile for the given domain (what the profile may restrict or require) and the manner of integration with systems in that domain — these two aspects are closely related and are deliberately discussed together.

Across all of the domains below, the following common principle applies: integration with an external system MUST NOT depend on a particular protocol, vendor, or technology; location information exchanged between systems MUST preserve the meaning defined by the LEM semantic model; and the technical constraints of an external system never constitute an extension of the LEM model (see Sections 3.1, 4.5). If an external system requires information that has no counterpart in LEM, that information MUST be treated as additional data or as an element of a profile, in accordance with the principles of this chapter.

4.3.1. Network infrastructure

The network-infrastructure profile defines the application of the LEM model to describe the physical location of technical infrastructure elements, such as access points, switches, IoT

devices, building-automation systems, and other elements requiring unambiguous spatial identification.

Integration with network systems may cover devices and systems responsible for handling technical infrastructure — LEM in this case serves as a shared semantic layer for the physical-location information of devices, but does NOT DEFINE device-discovery methods, network-configuration management, or network-communication mechanisms.

A profile MAY specify requirements regarding the minimum scope of location information required for the management of technical infrastructure.

4.3.2. Infrastructure and CMDB management

The infrastructure and CMDB management profile defines the application of the LEM model in inventory, asset-tracking, and technical-documentation systems — covering asset monitoring, configuration management, infrastructure maintenance, and the automation of operational processes.

An inventory system MAY store additional information specific to its own domain, but this information MUST NOT change the interpretation of the LEM components. LEM does NOT REPLACE infrastructure-management systems — it defines only a way of unambiguously describing the meaning of location information, enabling interoperability between systems with different data structures.

4.3.3. Emergency location

The emergency-location-services profile defines the application of the LEM model in systems that require the transmission of location information for safety purposes or for handling events requiring the determination of physical position.

LEM does NOT DEFINE emergency services, public-safety procedures, mechanisms for handling emergency reports, or methods of determining location, and does NOT REPLACE existing standards and systems designed to support emergency services. The role of LEM is to represent and interpret the resulting information supplied by a location system, in accordance with the principles of the semantic model set out in Chapter 2.

A profile MAY specify the minimum scope of information required to unambiguously determine location in a given environment. Given the importance of this domain, such a profile should give particular consideration to the privacy principles set out in Sections 6.3, 6.6, and 6.7.

4.3.3.1. Example profile elaboration (informative)

Below is a single, fully elaborated example of the “Emergency location” profile — from requirements, through the formal schema, to tested conformant and non-conformant examples. This is purely illustrative; the actual profile for a given deployment should be established by the entity responsible for that environment (see Section 4.1).

Component requirements (in accordance with the “Unambiguity of requirements” principle, Section 4.2):

Component	Status in the profile	Rationale
Building	Required	The response team must know which building to go to.
Level	Required	The storey is critical for navigation in multi-storey buildings.
Reference Space	Required	The room/reference space is the minimum unit of location that is operationally useful.
Site	Optional	Relevant only in multi-campus organizations; if present, it increases unambiguity.
Relative Position	Optional (recommended)	Increases precision within the Reference Space, but is not essential for undertaking an intervention.

Table 4.1. Component requirements for the emergency-location profile

The statuses in the table define the data completeness required by the profile. They do not define how unknown representation extensions are handled.

Formal schema (JSON Schema, informative), extending the base model’s structural skeleton (Section 2.5.6) via `allOf`:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://w3id.org/lem/1.0/profile/emergency",
  "title": "LEM – Profile: Emergency location (see Section 4.3.3)",
  "allOf": [
    { "$ref": "https://w3id.org/lem/1.0/schema/core" }
  ],
  "required": [
    "building",
    "level",
    "referenceSpace"
  ]
}
```

Example conformant with the profile:

```
{
  "site": "Main Campus",
  "building": "Engineering Center",
  "level": {
    "kind": "numeric",
    "value": 2
  },
  "referenceSpace": "Room201",
  "relativePosition": "East side, near window"
}
```

Example non-conformant with the profile (yet conformant with the base LEM model):

```
{
  "site": "Main Campus",
  "building": "Engineering Center"
}
```

Verification note. The above “non-conformant” example was deliberately chosen to illustrate the rule in Section 4.4.3: when validated against the base model’s structural

skeleton (Section 2.5.6), this document gave the result VALID — it is a fully valid LEM description (a partial description; see Section 2.4.4). The same document, when validated against the profile schema, gave the result INVALID, with two specific errors: the missing `level` and `referenceSpace` fields. This is a direct confirmation of the principle stated in Section 4.4.3: “Information may be a valid representation of LEM while simultaneously failing to satisfy the requirements of a particular application profile.”

4.3.4. Indoor positioning systems

The indoor-positioning-systems profile defines the application of the LEM model in indoor location environments, in which position is determined relative to reference spaces and spatial relations.

The profile does NOT DEFINE positioning technologies or measurement methods. It specifies only how location information is expressed in accordance with the LEM model. LEM does NOT DEFINE the measurement methods or positioning algorithms used by the systems that supply the input data.

4.3.5. Technical documentation

LEM MAY be used to create consistent location descriptions for infrastructure elements in technical documentation, inventory systems, and infrastructure-maintenance support tools.

Information conformant with LEM MAY be stored or presented in various documentation systems while preserving the same semantic interpretation. LEM does NOT DEFINE documentation formats, documentation tools, documentation-creation methods, or change-management processes for technical documentation.

4.3.6. Automation and orchestration

LEM MAY be used as a semantic layer for automation and orchestration systems that require access to location information for infrastructure elements. Location information conformant with LEM MAY be used by external systems as input data for carrying out automated processes.

LEM does NOT DEFINE automation mechanisms, control rules, orchestration logic, or the manner in which location information is used by execution systems.

4.3.7. Other domains

The LEM model MAY be applied in other domains requiring an unambiguous representation or exchange of location information (e.g., building-automation systems, security systems, spatial-documentation systems, analytical systems, resource-orchestration systems), provided that the manner of using the model remains consistent with the principles specified in this document.

New applications MUST NOT change the meaning of the LEM model’s components or define an alternative interpretation of location information. Requirements specific to a given domain SHOULD be defined through LEM profiles, in accordance with the principles of this chapter.

4.4. Constraints and implementation requirements of profiles

Profiles may specify implementation requirements arising from the specific character of a given application domain. These requirements concern the manner of using the LEM model and do NOT CONSTITUTE an extension of its base semantics.

4.4.1. Minimum scope of information

A profile MAY specify a minimum set of components required to support location information in accordance with a given application. The minimum scope of information specified by a profile does NOT CHANGE the meaning of the LEM model's components.

4.4.2. Required components

A profile MAY specify components of the model whose presence is required in a given application. A requirement that a component be present arises from the needs of a specific profile and does NOT MEAN that the component is mandatory in the base LEM model.

4.4.3. Profile validation rules

Profile validation is the process of verifying the conformance of a location description with the requirements specified by a given profile.

Profile validation is independent of the base semantic interpretation of the LEM model. Information may be a valid representation of LEM while simultaneously failing to satisfy the requirements of a particular application profile.

4.4.4. Application-specific data schemas

Profiles MAY define technical data schemas, such as JSON Schema, field types, value constraints, validation rules, and implementation requirements.

These schemas specify how the LEM model is used in a specific environment and do NOT CONSTITUTE an extension of the LEM semantic model.

The following JSON Schema skeleton is purely informative and illustrates how a profile can formally express the requirements described in prose in Sections 4.4.1–4.4.2:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://w3id.org/lem/1.0/profile/network-infrastructure",
  "title": "Example profile: Network infrastructure",
  "allOf": [
    { "$ref": "https://w3id.org/lem/1.0/schema/core" },
    {
      "type": "object",
      "required": ["building", "referenceSpace"],
      "not": { "required": ["relativePosition"] }
    }
  ]
}
```

Conformance of data with the above schema constitutes profile-conformance validation (see Sections 4.4.3, 5.3) and is independent of the validation of base semantic conformance with the LEM model specified in Section 2.5.

4.5. Conformance of profiles with the LEM model

Every profile **MUST** remain conformant with the LEM semantic model specified in Chapter 2.

A profile **MUST NOT**:

- change the interpretation of the model's components,
- define alternative semantics for location information,
- introduce new semantic components,
- treat the technical constraints of an external system as grounds for extending the LEM model.

Conformance with LEM is independent of conformance with the requirements of a particular profile. A system **MAY** be a valid representation of LEM information even if it does **NOT SATISFY** the requirements of a given application profile.

4.6. Extending and versioning profiles

LEM profiles may evolve over time in response to changing domain requirements. The evolution of profiles **MUST NOT** compromise the semantic stability of the base LEM model. The general rules for classifying model changes (corrective / extending / semantic), specified in Chapter 7, apply, mutatis mutandis, to the evolution of profiles.

4.6.1. Profile inheritance

Profiles **MAY** create specializations of existing profiles. A child profile **MAY** extend the requirements of a parent profile, but **MUST NOT**: change the meaning of inherited components, remove interpretations specified by the parent profile, or violate constraints arising from the parent profile.

4.6.2. Backward compatibility

The evolution of a profile **MUST NOT** lead to a change in the interpretation of components used in earlier versions of the profile. Changes to a profile should allow existing implementations to remain conformant, wherever the base semantics of the LEM model do not change.

4.6.3. Evolution of domain profiles

Domain profiles **MAY** be updated in response to the changing needs of application environments. An update to a profile **MUST** preserve: conformance with the LEM semantic model, the stability of the interpretation of existing components, and the ability to unambiguously determine the differences between versions of the profile.

The evolution of profiles **MUST NOT** require changes to the base LEM semantic model, if the need for change arises solely from the requirements of a specific application domain.

5. Implementation Considerations

5.1. Processing representations

A LEM implementation **MUST** allow data recorded in a LEM representation to be mapped onto the components of the semantic model. The processing procedure **MUST NOT** change the interpretation of the components specified in Chapter 2. This also applies to partial descriptions (see Section 2.4.4) — the absence of a component **MUST** be mapped as the absence of information at a lower level of precision, and not interpreted as an error. The rules for handling elements not recognized by an implementation are specified in Section 7.7.

5.2. Internal implementation representation

An implementation **MAY** use internal data structures different from the interchange representation. The internal representation **MUST NOT** affect the semantics of LEM information.

5.3. Data validation

An implementation **MAY** perform validation in the following areas:

- **syntactic correctness** — conformance of the encoding with the structure required by the representation,
- **conformance with representation rules** — the correctness of the mapping of representation elements onto the model's components (this includes assessing the data completeness required by a given representation),
- **conformance with the application profile** — satisfaction of the requirements of a particular profile (see Section 4.4.3).

Validation **MUST NOT** define a new interpretation of the information.

5.4. Error handling

An implementation **MUST** distinguish error categories corresponding to the validation areas in Section 5.3:

- representation-syntax errors,
- representation-rule-conformance errors,
- profile-conformance errors.

The technical means of signaling these error categories (error codes, reporting mechanisms, etc.) remain outside the scope of the LEM model.

Invalid or ambiguous data **MUST NOT** be interpreted as valid LEM information — this is a direct consequence of the unambiguity-of-interpretation principle specified in Section 2.5.1.

5.5. Versioning and implementation compatibility

Implementation changes **MUST NOT** cause a change in the meaning of the LEM model's components.

Implementations **SHOULD** allow the version of the representations used to be identified, and **SHOULD** retain the ability to process data conformant with earlier versions of the model (see Chapter 7).

6. Security and Privacy Considerations

6.1. Scope of the model's responsibility

The LEM model defines exclusively the semantics of location information and the rules for its interpretation. LEM does NOT DEFINE authentication, authorization, access-control, encryption, transmission-protection, data-retention, or location-information-management mechanisms.

Implementations using LEM SHOULD provide data-protection mechanisms appropriate to the requirements of the environment in which the model is applied. The LEM model neither defines nor verifies the satisfaction of this requirement — it remains the responsibility of the implementation and the environment, similarly to other non-model requirements specified in Section 2.5.5.

The security of location information depends on how it is acquired, stored, processed, and disclosed by the systems that use the LEM model.

6.2. Protecting information integrity

Location information represented in accordance with LEM MUST remain conformant with the canonical interpretation of the model.

An implementation MUST NOT modify the meaning of location components in a way that changes their interpretation as specified in Chapter 2. Any modification of location information that violates the semantic principles specified in the LEM model invalidates the conformance of that data with the model.

Mechanisms that ensure data integrity, including change control, signing, checksums, or other data-protection methods, remain the responsibility of the implementation.

6.3. Separation of location from identification

The LEM model describes information about physical location independently of the identification of an object, user, or system.

LEM does NOT DEFINE:

- object-identification mechanisms,
- user-identification mechanisms,
- ways of linking location to a specific person or entity,
- identity-management rules.

Linking location information to identifiers of objects or persons is carried out outside the LEM model and is subject to the rules in force within the given system. Identifiers assigned by external systems may be used by LEM implementations solely as a means of referring to an element of a location description (see Appendix A.9) — such use does not constitute an

identification mechanism within the meaning of the LEM model, nor does it weaken the separation principle specified in this section.

Implementations **MUST** take into account that combining location information with other data may give rise to additional security or privacy requirements.

6.4. Trust in the source of location information

The LEM model does not specify methods of acquiring location information or ways of assessing the reliability of a data source.

LEM does NOT DEFINE:

- location-measurement technologies,
- position-determination algorithms,
- methods for assessing measurement accuracy or quality,
- mechanisms for confirming data provenance.

An implementation using LEM **MAY** store additional information concerning the provenance or quality of the data; however, this information does not form part of the base semantics of the LEM model unless specified by an appropriate profile conformant with the principles of Chapter 4.

6.5. Security profiles

Security requirements specific to particular environments **MAY** be defined through security profiles or application profiles conformant with the principles specified in Chapter 4.

A security profile:

- **MUST NOT** change the meaning of the LEM model's components,
- **MUST NOT** define an alternative interpretation of location information,
- **MAY** specify requirements regarding the protection, access, or manner of use of the information.

Security profiles remain constraints on the implementation or application of the model, not an extension of LEM's semantics.

6.6. Privacy of location information

The LEM model is neutral with respect to the legal or organizational classification of location information.

LEM does NOT DETERMINE whether particular location information constitutes personal data, confidential information, or any other category of protected data.

Assessing the character of the information and applying appropriate protection mechanisms remains the responsibility of the systems using the LEM model.

Implementations **MUST** take into account privacy requirements arising from applicable regulations, organizational policies, and the nature of the data being processed. In particular, implementations **SHOULD** take into account that even a limited subset of LEM components (e.g., Reference Space and Relative Position without the full hierarchical context) may, depending on the context, constitute information of material significance to privacy (see Section 2.4.4).

6.7. Minimizing disclosed information

Systems using LEM **SHOULD** limit the scope of disclosed location information to the level required by the specific purpose of the application.

The disclosed representation of location information **MAY** use a restricted subset of the model's components, provided that conformance with the requirements of the given profile and with the semantic interpretation of LEM is preserved.

The LEM model does not define disclosure policies or access-control mechanisms. These rules remain the responsibility of the implementation.

6.8. Semantic consistency of extensions and security

Extensions using the LEM model **MUST** remain conformant with the base semantics of the model (see Sections 3.1, 4.5, 7.4).

An extension **MUST NOT** introduce new meanings for existing components or cause ambiguity in the interpretation of location information.

From a security standpoint, an extension that fails to satisfy this principle constitutes an additional risk: an extension field mistakenly interpreted as a standard LEM component may lead an implementation to an incorrect conclusion about an object's location. Implementations **SHOULD** clearly distinguish extension data from the base components of the LEM model during processing (see Sections 5.1, 7.7).

Additional information required by a particular application environment **SHOULD** be defined through extension mechanisms or profiles, and not through a change to the base LEM semantic model.

6.9. Consolidated threat model (informative)

The table below consolidates, in a single place, the threats addressed by the principles distributed across Sections 6.1–6.8, together with the corresponding defense mechanisms — which, in accordance with Section 6.1, remain outside the scope of the LEM model. It is informative and illustrative in character, and does not constitute an exhaustive, formal threat analysis.

Items 6 and 7 follow directly from properties of the LEM model described in other chapters (respectively: the distinction between extensions and base components, and the independence of profile validation from base-model validation), and illustrate that the

model's semantic discipline also has practical consequences for security, and not only for interpretive consistency.

No.	Threat	Vector	Consequence	Defense mechanism (outside LEM scope)	Ref.
1	Transmission eavesdropping	Interception of location data during transfer between systems	Disclosure of location to unauthorized parties	Transport encryption (e.g., TLS)	6.1
2	In-transit integrity violation	Undetected modification of component values	Incorrect interpretation of location — particularly critical in the emergency-location profile (4.3.3)	Signing, checksums	6.2
3	Data-source spoofing	Injection of data from a spoofed or untrusted source	Trust placed in false location information	Source authentication	6.4
4	Deanonimization through correlation	Combining location information with an external identifier or pattern	Disclosure of a person's identity despite formal location↔identity separation	Access control, restriction of dataset linkage	6.3, 6.6
5	Excessive disclosure of precision	Disclosure of a full, precise location description where the purpose required lower precision	Disclosure of more information than necessary for the purpose	Minimization of the scope of disclosed components	6.7, 2.4.4
6	Injection of an unrecognized extension	Addition of a field whose name/structure mimics a LEM component	Incorrect interpretation of extension data as part of the semantic model	Explicit, technical distinction between extensions and base components	6.8, 5.1
7	Bypassing profile requirements	Deliberate submission of a description conformant with the base model but not satisfying profile requirements	Action taken on the basis of insufficiently precise information	Enforcement of profile-conformance validation on the recipient side	4.4.3, 4.3.3.1
8	Use of stale data	Receipt and use of an outdated state of location information	Action taken on the basis of an object's outdated location	Timestamps, data-freshness mechanisms	A.12, 6.4

Table 6.1. Consolidated threat model (informative)

7. Evolution, Extension, and Versioning of the LEM Model

7.1. Classification of model changes

Changes concerning the LEM model **MUST** be classified according to their impact on the model's semantics. The classification below concerns changes to the base model; the rules for the evolution of profiles themselves are specified in Section 4.6, which refers back to this classification as appropriate.

The following types of change are distinguished:

7.1.1. Corrective change

A corrective change removes ambiguities or documentation errors, or improves the description of existing elements of the model.

A corrective change **MUST NOT** change the interpretation of the LEM model's components or affect conformant implementations of existing versions.

7.1.2. Extending change

An extending change introduces new capabilities to the model while preserving existing semantics.

An extending change **MAY** include: adding new components, adding new optional properties, extending the range of supported representations, adding new application profiles.

An extending change **MUST NOT** cause a change in the meaning of the LEM model's existing components.

7.1.3. Semantic change

A semantic change affects the base meaning of elements of the LEM model.

Examples of semantic changes include: changing the definition of an existing component, changing a relation between components, changing the rules of canonical interpretation, changing the rules for determining semantic equivalence.

A semantic change **MUST** be treated as a change to the base version of the LEM model.

7.2. Versioning principles

LEM versioning **MUST** allow the compatibility between implementations using the model to be unambiguously determined.

Versioning should distinguish between: the version of the LEM semantic model, the version of the representation rules, and the version of application profiles.

A change in the version of a representation or profile **MUST NOT** cause a change in the base semantics of the LEM model.

7.3. Version compatibility

LEM implementations **MUST** specify the version of the semantic model according to which location data is interpreted.

Two implementations are semantically compatible if:

- they use a compatible version of the LEM semantic model,
- they interpret the components in accordance with the same semantic model,
- they preserve the rules of canonical interpretation.

Differences in representation or profile versions may be handled independently of the core semantic version, provided that interpretive conformance is preserved.

7.4. Principles for extending the model

Extensions to the LEM model **MUST** preserve the existing semantics of the base components.

A new component or property:

- **MUST NOT** change the meaning of existing components,
- **MUST NOT** cause an alternative interpretation of existing data,
- **MUST** have an unambiguously defined meaning.

If a new meaning for location information needs to be introduced, the change **MUST** be made in the base LEM model, not through a profile or a local implementation.

This section concerns extensions intended for formal incorporation into the LEM model. Extensions of a local nature, not subject to such incorporation, are described in Section 7.5.

7.5. Private and domain-specific extensions

Implementations **MAY** define local or domain-specific extensions, intended solely for use within a particular environment and not subject to formal incorporation into the base model, provided that such extensions:

- do not change the meaning of components defined by LEM,
- do not cause ambiguous interpretation of data conformant with LEM,
- are clearly separated from the base LEM model.

Local extensions **MUST NOT** be treated as standard LEM components without their formal incorporation into the model in accordance with the principles of Section 7.4.

7.6. Process for semantic changes

Changes affecting the base semantics of the LEM model **MUST** be subject to a special versioning procedure.

This procedure **MUST** ensure: identification of the scope of the change, determination of its impact on existing implementations, definition of migration rules, and preservation of the ability to interpret older data.

A semantic change **MUST NOT** be introduced solely through a change to a representation or an application profile.

The manner in which this process is carried out in practice (e.g., decision-making body, consultation procedure, publication of changes) remains outside the scope of the LEM model.

7.7. Handling unknown elements

LEM implementations **MUST** distinguish unknown elements according to their significance for data interpretation. Whether a given element is, for a given implementation, optional or critical in character is determined contextually — on the basis of the requirements of the application profile (see Sections 4.2, 4.4.2) — and not imposed top-down by the LEM model itself.

7.7.1. Optional element

An optional element, the absence of which does not affect the interpretation of the base location information, **MAY** be omitted by an implementation.

7.7.2. Critical element

A critical element, whose meaning is required for the correct interpretation of location information, **MUST NOT** be omitted.

An implementation that does not support a critical element **MUST** mark the data as non-interpretable or non-conformant with the supported version of the model (see Section 5.4).

7.8. Stability of the LEM model

The development of the LEM model **MUST** preserve the stability of its base semantics.

Changes to the model should favor: extension over redefinition, the addition of new capabilities over changes to existing meanings, and the preservation of compatibility for existing data.

The primary goal of the LEM evolution process is to preserve the ability to unambiguously interpret location information across successive versions of the model.

8. Related Work and Positioning of the Model

LEM concerns an area in which mature location standards and building models already exist. The purpose of this chapter is not to replace these solutions, but to delineate the scope boundary of the proposed model.

8.1. Location information standards

PIDF-LO defines a location object carried within a presence document and covers both geodetic and civic information [13, 16]. RFC 5491 further constrains the use of PIDF-LO to improve interoperable interpretation [17]. LEM has a narrower purpose: it does not define a presence document, a usage policy, or geometry, but rather a five-component location core and a set of representational mappings.

IndoorGML 2.0 is an extensive conceptual model of indoor space, topology, connectivity, and navigation [12]. IndoorLocationGML, in turn, proposes a framework for encoding absolute and relative location in indoor environments [19]. Compared with these solutions, LEM does not aspire to model geometry or routes; it can serve as a small location descriptor at the boundary of systems that store a richer model of space.

8.2. Semantic building models

Brick is a metadata ontology for portable smart-building applications, modeling, among other things, assets, spaces, and location relations [1]. Project Haystack defines tagging conventions covering the site, space, floor, and room hierarchy, as well as storey numbers [14]. Both solutions offer a richer description of assets and building relations. LEM focuses on a portable description of the entity's location itself, independent of the full ontology of the source system.

8.3. Scope comparison

Table 8.1 compares scope, not maturity or quality of the solutions. “Partially” denotes that a feature exists in a different structure or as part of a broader model.

8.4. Boundary of the claimed novelty

The claimed contribution of LEM consists in the combination of: a small five-component core, an exact canonical tuple, partial descriptions, profiles, and a set of mappings for networking and management environments. The document does not claim to introduce the first representation of indoor location, nor that it replaces PIDF-LO, IndoorGML, Brick, or Haystack.

Feature	LEM	PIDF-LO	IndoorGML	Brick	Haystack
Small, format-independent core	yes	no	no	no	no
Geometry / coordinates	mapping	yes	yes	partially	partially
Indoor topology and navigation	no	no	yes	partially	partially

Site/building/level/space hierarchy	yes	partially	yes	yes	yes
Textual relative-position description	yes	partially	yes	partially	partially
Partial descriptions and requirement profiles	yes	usage rules	extensions	extensions	conventions
Combined JSON/YANG/TLV mappings	yes	no	no	no	no
Strict canonical-form equality	yes	different model	different model	graph identity	conventions

Table 8.1. Scope comparison of LEM and selected related solutions

This version is a technical specification. The reference artefacts confirm the syntactic consistency of the schemas and examples, but do not yet constitute an experimental assessment of interoperability between independent implementations. Such an assessment remains a task for further research work.

Appendix A. Glossary of LEM Terms

A.1. Normative terms

The terms defined in this glossary carry normative meaning for the entire LEM specification. Use of a term defined in the glossary **MUST** be consistent with the meaning specified in this appendix.

The LEM model describes the meaning of location information, and not: the method of identifying objects, the method of acquiring location information, the method of determining position, the method of storing change history, or security or access-management mechanisms.

Extensions, profiles, and implementations of LEM **MUST NOT** introduce new meanings for existing terms or components of the model. Where a new semantic meaning needs to be introduced, the change **MUST** be made in the base LEM model, in accordance with the versioning principles specified in Chapter 7.

A.2. Glossary of terms

The table below contains definitions of the key terms of the LEM model.

Table A.1. Glossary of LEM model terms

A.1. Location Information describing the position of an object with respect to one or more reference spaces. Location constitutes a semantic description of spatial, hierarchical, or other relations defined by the LEM model. LEM does not define measurement methods or ways of acquiring location information.

A.2. Object / Entity A physical or logical element described by the LEM model. The identity of an object is managed by external systems. LEM does not define mechanisms for creating, assigning, or managing object identifiers.

A.3. Component An element of the LEM model with a specific and unambiguous semantic meaning. The set of LEM model components is closed and defined by the model specification (see Section 2.1). Extensions may not change the meaning of existing components.

A.4. Reference Space An object or space possessing a reference role (see A.11) and serving as a point of reference for a location description. A reference space may represent any element of physical or logical space, provided it satisfies the interpretive requirements of the LEM model. The Site, Building, and Level components are named realizations of this role in a hierarchical context (see Section 2.1).

A.5. Relative Position Information specifying the position of an object with respect to a specified reference space or an element possessing a reference role. Relative position is interpreted in accordance with the rules of the LEM model and does not define a method of determining position.

A.6. Relation A directional semantic relationship between elements of the LEM model. A relation determines how the dependency between elements is interpreted. The direction of a relation is part of its meaning and must not be disregarded during interpretation of the information (see A.13).

A.7. Representation A form of recording information conformant with the LEM model, intended for storage, exchange, or processing of data. A representation may be textual, binary, or structural. Different representations are semantically equivalent if they lead to the same interpretation of information as defined by the LEM model, within the scope of the components they represent.

A.8. Canonical Interpretation The unambiguous meaning of information that follows from the rules of the LEM model, independent of the data representation used. The canonical interpretation determines how the information is to be understood, not its physical recording method.

A.9. Identification The process by which an external system assigns a designation or identifier to an object or space. LEM does not define mechanisms for identifying objects or reference spaces. Identifiers assigned outside the LEM model may be used by LEM implementations solely as a means of referring to elements of a location description — such use does not constitute an identification mechanism within the meaning of LEM (see Section 6.3).

A.10. Profile A restriction or specialization of the LEM model intended for a particular application domain. A profile may define requirements concerning the use of components and representations, but may not change the base semantics of the LEM model.

A.11. Reference Role The capacity of an object or model element to serve the function of a reference space within a particular interpretive context. The reference role does not change the base classification of an element, but determines how it is used within the location model. Site, Building, Level, and Reference Space are components that fulfill this role (see Section 2.1).

A.12. Location Information State The representation of location relations that hold at the moment of interpreting the information. The LEM model describes the state of location information and does not define mechanisms for managing change history, timestamps, or event logging.

A.13. Directed Relation A relation in which the order of the elements participating in the relation is part of its semantic meaning. The relation `located_in(Object, ReferenceSpace)` has a different meaning from the inverse relation `contains(ReferenceSpace, Object)` (see Section 2.2).

References

- [1] B. Balaji, A. Bhattacharya, G. Fierro, J. Gao, J. Gluck, D. Hong, A. Johansen, J. Koh, J. Ploennigs, Y. Agarwal, M. Berges, D. Culler, R. Gupta, M. B. Kjærgaard, M. Srivastava, and K. Whitehouse. Brick: Metadata schema for portable smart building applications. *Applied Energy*, 226:1273–1292, 2018.
- [2] M. Björklund. The YANG 1.1 data modeling language. RFC 7950, RFC Editor, 2016.
- [3] S. Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, RFC Editor, 1997.
- [4] T. Bray. The JavaScript Object Notation (JSON) data interchange format. RFC 8259, RFC Editor, 2017.
- [5] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, and T. Schaub. The GeoJSON format. RFC 7946, RFC Editor, 2016.
- [6] A. Costello. Punycode: A bootstring encoding of Unicode for internationalized domain names in applications. RFC 3492, RFC Editor, 2003.
- [7] D. Crocker and P. Overell. Augmented BNF for syntax specifications: ABNF. RFC 5234, RFC Editor, 2008.
- [8] ITU-T. Information technology—ASN.1 encoding rules: Specification of BER, CER and DER. International Telecommunication Union, 2021.
- [9] JSON Schema. JSON Schema specification, Draft 2020-12, 2022.
- [10] B. Leiba. Ambiguity of uppercase vs lowercase in RFC 2119 key words. RFC 8174, RFC Editor, 2017.
- [11] P. Mockapetris. Domain names—implementation and specification. RFC 1035, RFC Editor, 1987.
- [12] Open Geospatial Consortium. OGC IndoorGML 2.0. Open Geospatial Consortium, 2025.
- [13] J. Peterson. A presence-based GEOPRIV location object format. RFC 4119, RFC Editor, 2005.
- [14] Project Haystack. Spaces, 2026. Project Haystack 4.0 documentation, accessed 12 Sept. 2026.
- [15] H. Schulzrinne. DHCPv4 and DHCPv6 option for civic addresses configuration information. RFC 4776, RFC Editor, 2006.
- [16] M. Thomson and J. Winterbottom. Revised civic location format for PIDF-LO. RFC 5139, RFC Editor, 2008.

- [17] J. Winterbottom, M. Thomson, and H. Tschofenig. GEOPRIV presence information data format location object (PIDF-LO) usage clarification, considerations, and recommendations. RFC 5491, RFC Editor, 2009.
- [18] F. Yergeau. UTF-8, a transformation format of ISO 10646. RFC 3629, RFC Editor, 2003.
- [19] Qing Zhu, Yun Li, Qing Xiong, Sisi Zlatanova, Yulin Ding, Yeting Zhang, and Yan Zhou. IndoorLocationGML—a GML-based encoding framework for indoor location and navigation. *ISPRS International Journal of Geo-Information*, 5(12):220, 2016.